

Fiche de révision — Oral E6 · ColorRoom · Téo Trompier (E2)

Format : 20 min présentation (PDF) → 20 min démonstration (app) → 20 min questions.

Objectif : connaître le projet ET le code. Tout ci-dessous est tiré du dépôt réel

[NewGenesisAgency/color_room](#) .

1. Pitch (30 secondes, à savoir par cœur)

La **ColorRoom** est un équipement scientifique du laboratoire **ENTPE / LTDS / BPMNP**, hébergé à **LUMEN – La Cité de la Lumière** (Lyon Confluence) : **2 cellules jumelles, 42 plaques lumineuses** pilotées chacune par **32 canaux** couvrant tout le spectre visible. Le logiciel de recherche étant trop complexe pour l'initiation, j'ai développé une **application web de serious games**, embarquée sur **Raspberry Pi 5, 100 % hors-ligne**. Ma partie (**E2**) couvre la base de données, l'API, l'interface, les jeux solo et multijoueur, la mesure colorimétrique et la documentation.

2. Les faits à connaître par cœur

- **Matériel** : 42 plaques · 32 canaux/dalle · **18 LED spectrales (404 → 780 nm) + 14 LED phosphore** (blanc chaud) · 2 cellules jumelles.
- **Pile** : Next.js **16.2.1** (App Router), React **19.2.4**, TypeScript **5.5.3** (strict), better-sqlite3 **11.5.0**, Three.js **0.160.1**, Docker (multi-stage arm64), Portainer, port **8080**, Raspberry Pi **5**.
- **Équipe (8)** : E1 Bonnevey (infra/Docker/CI-CD), **E2 Trompier = moi** (BDD, API, UI, jeux, multi, doc), E3 Arbadji (éditeur), E4 Akyuz (tests API). 2 sous-équipes : JavaScript et Python.
- **Partenaires** : LUMEN · ENTPE/LTDS/BPMNP · contacts Labayrade & Vella · prof Delbosc.

3. Démonstration minutée (20 min)

Prépare une **vidéo de secours** de chaque étape (au cas où le matériel/Pi bug).

1. **Connexion / inscription** (2 min) — créer un apprenant, montrer la détection de rôle, la connexion auto.
2. **Catalogue + un jeu solo** (4 min) — lancer **Color Speed** : montrer les dalles s'allumer en temps réel (3D + vraies dalles), le score.
3. **Puissance 4 contre l'IA** (4 min) — choisir un niveau (Novice → Légendaire), montrer que l'IA bloque une menace (anti-piège), parler du minimax.
4. **Multijoueur** (3 min) — Spectre Chromatique : 2 terminaux rejoignent, projection d'une couleur, scores.

5. **Mesure CS-160** (3 min) — connecter, mesurer une dalle, lire X Y Z / Lv / x,y, point sur le diagramme CIE.
6. **Tableau de bord enseignant** (2 min) — classes, scores, export CSV.
7. **Coulisses** (2 min) — Portainer (conteneurs), le `docker compose`, la base SQLite (fichier).

4. Le code à connaître (fichiers clés)

- `lib/db/index.ts` — connexion SQLite **singleton** (`dbSingleton`), `migrate()` : `PRAGMA journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON` ; tables `crg_*` créées en `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés ; `seedAdminFromEnv()` (compte admin via `.env`).
- `lib/auth.ts` — `hashPassword` / `verifyPassword` (PBKDF2-HMAC-SHA512, 100 000 itérations, sel 16 o, clé 64 o, format `sel:hash`) ; `createSession` (token `randomBytes(32)`, **30 jours**), `renewSession` (fenêtre glissante).
- `app/api/auth/register/route.ts` — inscription en **transaction atomique** (`db.transaction`).
- `app/api/auth/login/route.ts` — `verifyPassword` + cookie `crg_session` **HttpOnly** + **SameSite=lax** (30 j).
- `app/api/supervision/batch/route.ts` — proxy LED : **sémaphore** `HW_CONCURRENCY=2` + file d'attente (supervision.exe est quasi-série) + `AbortController` (timeout).
- `app/_components/GamePuissance4.tsx` — IA **minimax** + **élagage alpha-bêta**, `scoreWindow` (heuristique fenêtres de 4), profondeurs **1/2/5/9/12**, anti-piège (`winningCols`).
- `app/_components/Room3D.tsx` — vue 3D Three.js (WebGLRenderer, `forceContextLoss` au démontage, pause via Page Visibility API).
- `lib/multiplayer.ts` + `app/api/multiplayer/state/route.ts` — sessions persistées en base (`crg_mp_sessions.state.json`), lues en **polling**.

5. Fiche réponses du jury (Q&A)

Projet & cahier des charges

- « **À quoi sert la ColorRoom ?** » → Reproduire des éclairages à spectres précis pour la recherche sur la perception des couleurs ; mon appli la rend accessible en initiation via des jeux.
- « **Pourquoi des serious games ?** » → Le logiciel de recherche est trop technique ; le jeu rend la lumière/couleur compréhensible pour des apprenants.
- « **Qu'as-TU fait précisément ?** » → Partie E2 : BDD (7+ tables), API, UI/design, jeux solo + Puissance 4 (IA) + multijoueur, mesure CS-160, documentation. L'éditeur no-code est la partie E3.

Architecture & Next.js

- « **Pourquoi Next.js ?** » → Un seul projet front (React) + back (**Route Handlers**), un seul runtime Node à déployer sur le Pi ; **Server Components** pour le rendu, App Router.

- « **Architecture ?** » → Navigateur → routes API Next.js → couche **lib/** (auth, db, services) → SQLite + matériel (proxy supervision, pont CS-160). Tout conteneurisé.

Choix techniques (la question Node-RED)

- « **Pourquoi pas Node-RED comme proposé ?** » → Node-RED est **flow-based** : excellent pour automatiser des flux, mais inadapté à une vraie UI multi-pages (3D, catalogue, jeux), et ses flux JSON sont durs à versionner. React/TS/Next donnent des composants réutilisables, un **typage strict** (erreurs à la compilation), du code versionnable.
- « **Pourquoi TypeScript ?** » → Typage statique : un champ manquant ou une couleur mal formée est détecté par **tsc** /ESLint, pas en production.

Réseau / Docker / Raspberry Pi

- « **Comment déployes-tu ?** » → Image **Docker multi-stage** (deps → builder → runner) pour **arm64**, **docker compose up -d --build**, supervision par **Portainer**, app sur le port 8080.
- « **Et le réseau ?** » → Wi-Fi local **ColorRoom_WiFi** fourni par le Pi ; les tablettes/téléphones se connectent en local ; le colorimètre est en **USB** sur le Pi.
- « **Hors-ligne, vraiment ?** » → Oui : app + SQLite + matériel, tout local. (L'IA cloud Gemini est optionnelle ; un modèle local Ollama peut prendre le relais.)

Base de données / SQLite

- « **Pourquoi SQLite ?** » → Base **embarquée** (un simple fichier), zéro serveur à installer, idéale sur Pi ; via **better-sqlite3** (API **synchrone**, requêtes **préparées** = anti-injection).
- « **SQLite tient la charge ?** » → Pour quelques dizaines d'utilisateurs locaux, oui. **WAL** autorise des lectures concurrentes pendant une écriture, **busy_timeout** évite les erreurs **SQLITE_BUSY**.
- « **Migrations ?** » → **CREATE TABLE IF NOT EXISTS** + **ALTER** protégés : **idempotentes**, une base neuve se crée seule, une existante se met à jour sans casser.

Variables & persistance (questions « pointues »)

- **Atomique / transactionnelle** → **db.transaction()** (better-sqlite3) enveloppe **BEGIN/COMMIT/ROLLBACK** : l'inscription (user + adhésion classe) est **tout-ou-rien** (Atomicité ACID). Sinon : comptes « à moitié créés ».
- **Volatile (en mémoire)** → En JS il n'y a pas de mot-clé **volatile** ; je parle de l'**état runtime** stocké en **useRef** (ex. l'état des dalles pendant un jeu) : rapide, mais **non persistant**, perdu au rechargement — par opposition aux données **persistées** en SQLite.
- **Concurrente** → Le matériel (supervision.exe) est quasi-série ; un **sémaphore** **HW_CONCURRENCY=2** + file d'attente borne les appels simultanés.
- **Réactive** → **useState** : modifier la valeur déclenche le **re-rendu** ciblé de l'interface.
- **Environnement** → **process.env** (clé API, mot de passe admin) lue depuis **.env**, **hors Git**.
- **Persistante** → SQLite dans un **volume Docker** : survit aux redémarrages.

Sécurité / RGPD

- « **Comment stockes-tu les mots de passe ?** » → Jamais en clair : **PBKDF2-HMAC-SHA512**, 100 000 itérations, **sel aléatoire** 16 o, clé 64 o, format **sel:hash** . La vérification recalcule le hash et compare.
- « **Pourquoi PBKDF2 et pas bcrypt/argon2 ?** » → PBKDF2 est **natif** dans le module **crypto** de Node (aucune dépendance native à compiler sur arm64) ; 100 000 itérations donnent un coût correct. Argon2 serait plus moderne mais ajouterait une dépendance native.
- « **Les sessions ?** » → Token aléatoire (**randomBytes(32)**) en cookie **HttpOnly** (inaccessible au JS → anti-XSS) + **SameSite=lax** (anti-CSRF), **30 jours** avec **renouvellement glissant**.
- « **RGPD ?** » → Minimisation (un **pseudo** suffit, pas de nom/email) ; mots de passe hachés ; tout **local** (ENTPE) ; **droit à l'effacement** en cascade ; cloisonnement par rôle.

Jeux & temps réel

- « **Comment une dalle s'allume vite ?** » → Le navigateur ne parle pas directement au matériel : une **route proxy** relaie. Les **32 canaux** sont envoyés **en parallèle** (**Promise.all** , concurrence bornée) avec **timeout** (**AbortController**).
- « **La couleur à l'écran = la dalle ?** » → Oui, rendu **unifié** ; **CHANNEL_PROFILES** associe chaque canal à sa **longueur d'onde réelle**.

IA (Puissance 4)

- « **Explique ton IA.** » → **Minimax** : on simule l'arbre des coups, MAX pour l'IA / MIN pour l'adversaire ; **élagage alpha-bêta** coupe les branches inutiles ; **profondeur** limitée (1/2/5/9/12 selon le niveau). L'évaluation **scoreWindow** note chaque fenêtre de 4 cases : **défense pondérée plus fort que l'attaque** (-170 vs +130), victoire = **WIN_SCORE** (1 000 000), bonus de **centralité**. Anti-piège : l'IA ne donne pas une victoire immédiate à l'adversaire.
- « **Temps de réponse ?** » → Quasi instantané grâce à l'alpha-bêta et au poids central (exploration des bons coups en premier).

Multijoueur

- « **WebSockets ?** » → Non : l'état est **persisté en base** (**crg_mp_sessions.state_json**) et lu en **polling** de **/state** . Plus simple et robuste à déployer sur Pi, suffisant pour la cadence des jeux.
- « **Présence des joueurs ?** » → **Heartbeat** régulier ; jetons de session = **UUID** ; l'hôte (siège 1) démarre/arrête.

Mesure / colorimétrie

- « **Comment mesures-tu ?** » → Colorimètre **Konica Minolta CS-150/160** via un **pont .NET** exposé par **/api/cs160** . On lit le **tristimulus CIE XYZ**, la **luminance Lv (cd/m²)** et la **chromaticité (x, y)**, tracés sur le **diagramme CIE 1931**.
- « **ΔE ?** » → Écart de couleur entre la mesure et la cible, utilisé pour scorer la précision dans les jeux de mesure.

Qualité / tests / gestion

- « **Tests ?** » → Vérification de **types (tsc)** + **build de production** à chaque étape ; tests de l'API et fiches de recette côté E4 ; transactions ACID, exports UTF-8 + BOM.

- « **Gestion de projet ?** » → Git + GitHub (branches, commits clairs), développement incrémental, 15 diagrammes UML + guide technique.

UML

- « **Différence ERD / diagramme de classes ?** » → L'**ERD** modélise les **tables relationnelles** (clés primaires/étrangères) ; le **diagramme de classes** modélise la **conception objet** (TypeScript).
-

6. Questions pièges — réponses courtes

- « **C'est toi qui as tout fait ?** » → Non, projet d'équipe ; je présente **ma partie E2** (le cœur de l'appli), qui s'appuie sur l'infra de E1 et est testée par E4.
 - « **Et si une dalle tombe en panne ?** » → L'API gère par plaque, l'appli continue, le jeu ne plante pas.
 - « **Combien de joueurs simultanés ?** » → 1 dalle/joueur (jusqu'à 42) en local, limité par le Wi-Fi de la salle.
 - « **Pourquoi pas une vraie base serveur (PostgreSQL) ?** » → Surdimensionné pour un déploiement embarqué mono-poste hors-ligne ; SQLite suffit et simplifie l'install.
 - « **L'IA générative dans le projet ?** » → Hors dossier E2 ; c'est une **exploration personnelle** (génération de jeux assistée par IA locale Ollama) que je peux montrer en bonus.
-

Bon courage Téo — relis la section 5 deux fois, c'est 90 % des questions.